MATERI PRAKTIKUM PENGAYAAN: PEMBAHASAN MENDALAM STORED PROCEDURE, UDF, CURSOR & ERROR HANDLING

Modul pengayaan ini disusun secara terstruktur untuk melengkapi Materi Praktikum 4 (USER DEFINE FUNCTION & STORE PROCEDURE) pada mata kuliah Sistem Manajemen Basis Data Lanjut (SMBD Lanjut). Fokus utama modul ini adalah menjembatani pemahaman mahasiswa dari materi basis data dasar (DDL, DML, dan View) menuju paradigma pemrograman logika bisnis terkompilasi di sisi server (RDBMS). Modul dilengkapi penjelasan analogi dunia nyata, perbandingan perbedaan UDF vs Stored Procedure, manajemen parameter (IN, OUT, INOUT), struktur perulangan CURSOR baris per baris, penanganan eksepsi terstruktur (SIGNAL & HANDLER), studi kasus kasir 3NF, metodologi mengajar praktikum bagi asisten, serta 10 latihan soal bertingkat beserta kunci jawaban resmi.

1. JEMBATAN KONSEPTUAL: DARI DDL, DML, VIEW MENUJU STORED ROUTINE

Sebelum melangkah ke materi Stored Routine, mahasiswa telah menguasai tiga pilar fundamental SQL pada tingkat sebelumnya:
• DDL (Data Definition Language): Digunakan untuk merancang dan mendefinisikan struktur fisik tabel (CREATE TABLE, ALTER, DROP).
• DML (Data Manipulation Language): Digunakan untuk memanipulasi baris data di dalam tabel fisik (SELECT, INSERT, UPDATE, DELETE).
• View: Merupakan kueri SELECT yang disimpan dengan nama di dalam katalog basis data (*virtual table*), yang bertujuan memudahkan pembacaan tanpa menduplikasi data fisik.

Materi Stored Routine adalah satu lompatan besar di atas ketiga konsep tersebut. Jika pada View kita hanya dapat menyajikan proyeksi data dari satu perintah SELECT pasif, maka Stored Routine memungkinkan kita mengeksekusi serangkaian algoritma lengkap: percabangan kondisi, perulangan loop, variabel internal, serta mutasi transaksi pada banyak tabel secara serentak.

> 🔑 **Kalimat Kunci Pemahaman:** *"Kalau VIEW itu bertugas MENYIMPAN QUERY (tampilan data pasif), maka STORED ROUTINE itu bertugas MENYIMPAN PROGRAM (logika bisnis aktif di dalam database)."*

> 💡 **Analogi Skenario Kasir Minimarket:** *Bayangkan operasional kasir minimarket. Setiap kali ada pelanggan yang berbelanja, sistem kasir harus melakukan rangkaian langkah: (1) cek sisa stok produk di gudang -> (2) kurangi stok fisik barang -> (3) buat nomor nota baru -> (4) cetak item nota -> (5) hitung poin reward member.*
> *Jika langkah-langkah ini ditulis ulang di tiap aplikasi klien (aplikasi POS kasir, aplikasi mobile e-commerce, portal web admin), maka jika terjadi perubahan aturan bisnis (misalnya diskon baru atau skema poin), kode di semua aplikasi klien harus diubah satu per satu. Hal ini berisiko menimbulkan ketidaksinkronan aturan dan celah kecurangan (fraud).*
> *Solusinya: Tulis seluruh urutan langkah tersebut satu kali saja di dalam Stored Routine database. Semua aplikasi klien cukup 'memanggil' subrutin tersebut.*

2. ARSITEKTUR & EMPAT KEUNTUNGAN LOGIKA SISI SERVER

Pada sistem basis data relasional (RDBMS) seperti MySQL, MariaDB, Oracle, atau PostgreSQL, Stored Routine adalah blok instruksi SQL terkompilasi yang disimpan secara permanen di dalam data dictionary (katalog sistem) dan dieksekusi langsung oleh server database engine. Memindahkan logika bisnis dari aplikasi klien langsung ke RDBMS memberikan 4 keunggulan:

- **1. Reduksi Lalu Lintas Jaringan (Network Traffic Optimization):** Pada arsitektur konvensional, aplikasi klien harus mengirimkan belasan kueri SELECT dan UPDATE secara bolak-balik melalui jaringan kabel/WiFi. Dengan Stored Routine,

aplikasi cukup mengirimkan satu baris perintah CALL, dan seluruh rangkaian kueri dieksekusi langsung di RAM server, mengurangi latensi jaringan.

- **2. Sentralisasi Logika Bisnis (Centralized Business Logic):** Aturan bisnis krusial (misal kalkulasi poin reward, diskon member, validasi batas minimum stok) didefinisikan satu kali secara terpusat di basis data. Seluruh aplikasi klien memakai aturan yang sama persis, menjaga konsistensi data.
- **3. Kompilasi dan Eksekusi Cepat (Execution Plan Caching):** RDBMS mem-parsing, memeriksa hak akses, dan mengompilasi kode subrutin pada eksekusi pertama, lalu menyimpan rencana eksekusinya (execution plan) di dalam cache memori buffer. Pemanggilan berikutnya berjalan secara instan tanpa proses kompilasi ulang.
- **4. Lapisan Keamanan & Isolasi Hak Akses (Security & Privilege Isolation):** Pengguna aplikasi (seperti kasir atau staf gudang) tidak perlu diberi hak akses langsung (INSERT/UPDATE/DELETE) pada tabel fisik master. Cukup berikan izin EXECUTE pada Stored Procedure tertentu. Dengan demikian, data master aman dari modifikasi liar.

3. PERBEDAAN UTAMA: FUNCTION (UDF) VS STORED PROCEDURE

Meskipun sama-sama merupakan blok kode prosedural SQL terkompilasi di sisi server, User Defined Function (UDF) dan Stored Procedure memiliki batasan arsitektural dan tujuan perancangan yang sangat berbeda:

> 💡 **Analogi Kalkulator vs Mesin Pekerja:** • *FUNCTION ibarat KALKULATOR: Kamu memasukkan angka atau data, dan ia PASTI mengembalikan tepat SATU hasil nilai kembalian (via klausa RETURN). Dipakai di dalam kueri SELECT biasa.*
> • *STORED PROCEDURE ibarat ROBOT / MESIN PEKERJA: Kamu memerintahkannya untuk menjalankan serangkaian tindakan atau pekerjaan nyata (memeriksa data, mengubah tabel, memotong stok, membatalkan transaksi). Dipanggil secara mandiri dengan perintah CALL.*

Parameter Komparasi	User Defined Function (UDF)	Stored Procedure
Nilai Kembalian (Return)	Wajib mengembalikan tepat satu nilai skalar tunggal melalui klausa RETURNS dan pernyataan RETURN.	Tidak mengembalikan nilai melalui klausa RETURN; dapat menghasilkan nilai lewat parameter bertipe OUT/INOUT atau menampilkan result set.
Cara Pemanggilan	Dipanggil di dalam ekspresi kueri SQL biasa (contoh: SELECT fc_name(), WHERE fc_name() > 0, ORDER BY fc_name()).	Dipanggil secara mandiri sebagai perintah terpisah menggunakan sintaks khusus: CALL proc_name(argumen);.
Manipulasi Data Fisik (DML)	Sangat dibatasi; secara standar tidak diperbolehkan melakukan operasi modifikasi data tabel fisik (INSERT/UPDATE/DELETE) saat dipanggil oleh kueri baca.	Mendukung eksekusi DML penuh secara bebas: dapat menyisipkan, memperbarui, menghapus baris tabel, serta mengontrol transaksi ACID (COMMIT/ROLLBACK).
Fokus Penggunaan	Kalkulasi matematis, manipulasi format teks, konversi mata uang, atau penentuan kategori/tier berbasis rumus murni.	Mengeksekusi alur kerja transaksi bisnis multi-tabel, batch processing berkala, dan otomasi checkout kasir.

4. USER DEFINED FUNCTION (UDF) DI MYSQL

MySQL telah menyediakan beragam fungsi bawaan seperti UPPER(), NOW(), FORMAT(), dan CONCAT(). Namun, saat pengembang membutuhkan fungsi kalkulasi khusus yang spesifik untuk bisnis mereka (seperti rumus diskon member bertingkat atau format mata uang rupiah), maka dibuatlah User Defined Function (UDF).

4.1 Sintaks Lengkap Deklarasi UDF

```
CREATE [DEFINER = user]
FUNCTION [IF NOT EXISTS] func_name ([param_name param_type, ...])
RETURNS return_type
[DETERMINISTIC | NOT DETERMINISTIC]
[CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA]
[SQL SECURITY { DEFINER | INVOKER }]
BEGIN
   DECLARE variabel_lokal tipe_data [DEFAULT nilai];
   -- Logika komputasi prosedural
   RETURN nilai_kembalian;
END;
```

4.2 Dua Hal Kritis yang Sering Ditanyakan Mahasiswa

Dalam sesi praktikum di laboratorium, mahasiswa sering kali mengalami kegagalan pada dua aspek fundamental berikut:

1. Mengapa Wajib Menggunakan DELIMITER $$? *Secara bawaan, MySQL menganggap tanda titik-koma (;) sebagai 'akhir dari satu perintah kueri'. Di dalam tubuh blok BEGIN...END suatu Function atau Procedure, terdapat banyak pernyataan logika yang masing-masing diakhiri dengan titik-koma (;). Jika delimiter tidak diganti, MySQL akan mengira perintah terputus pada titik-koma pertama, sehingga menghasilkan sintaks galat (Syntax Error).*
Oleh karena itu, sebelum menulis blok routine, kita ubah sementara tanda akhir perintah menjadi '$$' menggunakan perintah DELIMITER $$. Setelah blok ditutup dengan END$$, kita kembalikan lagi tanda akhir perintah ke titik-koma standar dengan DELIMITER ;
Analogi: Memberi tahu sistem bahwa tanda titik pada paragraf ini adalah tanda baca biasa, dan akhir dari seluruh cerita ditandai dengan tanda '$$'.

2. Makna Karakteristik DETERMINISTIC, NO SQL & READS SQL DATA: *Klausa karakteristik memberi tahu query optimizer dan sistem binary logging bagaimana memperlakukan fungsi tersebut:*
• *DETERMINISTIC: Jika diberi input yang sama, fungsi DIJAMIN selalu menghasilkan output yang sama (misal: perhitungan matematika atau pemformatan teks).*
• *NOT DETERMINISTIC: Fungsi dapat menghasilkan nilai berbeda meski input sama (misal fungsi yang membaca waktu NOW() atau membaca data tabel fisik yang dinamis).*
• *NO SQL: Menandakan bahwa fungsi murni kalkulasi matematis tanpa membaca atau menyentuh tabel sama sekali.*
• *READS SQL DATA: Menandakan fungsi membaca data tabel (memakai SELECT), tetapi tidak memodifikasi data tabel.*

4.3 Contoh Implementasi UDF: Generator Kode Nota & Kalkulasi Poin

```
DELIMITER $$

-- Fungsi 1: Generator Nomor Nota Unik Berbasis Tanggal (READS SQL DATA)
CREATE FUNCTION fcGenerateNoNota(p_tanggal DATE)
RETURNS CHAR(10)
DETERMINISTIC
READS SQL DATA
BEGIN
   DECLARE v_prefix CHAR(6);
   DECLARE v_last_no INT DEFAULT 0;
   DECLARE v_new_nonota CHAR(10);
```

```
    -- Format Prefix: YYMMDD (Contoh: 220803)
    SET v_prefix = DATE_FORMAT(p_tanggal, '%y%m%d');

    -- Cari nomor urut nota tertinggi pada tanggal bersangkutan
    SELECT IFNULL(MAX(CAST(RIGHT(nonota, 4) AS UNSIGNED)), 0)
    INTO v_last_no
    FROM nota
    WHERE LEFT(nonota, 6) = v_prefix;

    -- Format Hasil: YYMMDDXXXX (Contoh: 2208030001)
    SET v_new_nonota = CONCAT(v_prefix, LPAD(v_last_no + 1, 4, '0'));
    RETURN v_new_nonota;
END$$

-- Fungsi 2: Kalkulator Poin Belanja Kelipatan Rp10.000 (NO SQL)
CREATE FUNCTION fcHitungPoinBelanja(total_belanja INT)
RETURNS INT
DETERMINISTIC
NO SQL
BEGIN
    DECLARE jml_poin INT DEFAULT 0;
    IF total_belanja >= 10000 THEN
        SET jml_poin = FLOOR(total_belanja / 10000);
    END IF;
    RETURN jml_poin;
END$$

DELIMITER ;

-- Pengujian Eksekusi (Dipanggil langsung di dalam ekspresi SELECT):
SELECT fcGenerateNoNota('2022-08-03') AS 'Nomor Nota Baru';
SELECT fcHitungPoinBelanja(14800) AS 'Poin Pembelian (14.800)'; -- Output: 1
SELECT fcHitungPoinBelanja(45000) AS 'Poin Pembelian (45.000)'; -- Output: 4
```

5. STORED PROCEDURE: MANAJEMEN PARAMETER IN, OUT, DAN INOUT

Stored Procedure dirancang untuk menangani tugas transaksi yang kompleks dan multi-tabel. Salah satu keunggulan terbesar Stored Procedure adalah fleksibilitas parameternya, yang dapat diklasifikasikan ke dalam tiga mode interaksi data:

> 💡 **Analogi Tiga Mode Parameter: Mesin Fotokopi:** *Bayangkan mesin fotokopi di kantor administrasi:*
> • *Parameter IN ibarat LEMBAR ASLI yang kamu masukkan ke scanner: Mesin membaca isinya untuk diproses, namun kertas asli milikmu tidak akan berubah sama sekali.*
> • *Parameter OUT ibarat LEMBAR FOTOKOPI KELUAR: Awalnya kosong/NULL, lalu mesin mengisinya dengan hasil proses, dan lembaran tersebut kamu bawa pulang sebagai hasil akhir.*
> • *Parameter INOUT ibarat FORMULIR PENDAFTARAN: Kamu memasukkan satu lembar formulir berisi data awal, mesin/petugas menambahkan cap atau nomor registrasi baru pada lembaran tersebut, lalu mengembalikannya kepadamu.*

Mode Parameter	Arah Aliran Data	Karakteristik & Analogi Praktis

IN (Default)	Pemanggil -> Procedure	Nilai dikirim masuk ke dalam procedure sebagai input baca. Modifikasi variabel di dalam procedure tidak mempengaruhi variabel pemanggil.
OUT	Procedure -> Pemanggil	Variabel bernilai NULL saat masuk. Procedure melakukan kalkulasi dan mengisi nilai ke dalamnya untuk diambil kembali oleh pemanggil saat selesai.
INOUT	Pemanggil <-> Procedure	Variabel membawa nilai awal masuk, nilainya diperbarui di dalam procedure, dan nilai hasil akhirnya dikembalikan ke pemanggil.

📌 **Variabel Sesi Bertanda @:** *Untuk menampung nilai hasil dari parameter OUT dan INOUT, kita menggunakan variabel sesi yang diawali simbol '@' (misal: @kode_baru atau @poin_awal). Variabel sesi ini tersimpan selama koneksi aktif dan dapat diperiksa kapan saja menggunakan kueri SELECT @nama_variabel;.*

5.1 Contoh Implementasi Parameter IN, OUT, dan INOUT

```
DELIMITER $$

-- Procedure A: Parameter IN & OUT (Registrasi Member & Kembalikan ID Baru)
CREATE PROCEDURE spRegistrasiMember(
   IN p_nama VARCHAR(50),
   IN p_tgllahir DATE,
   IN p_jenkel CHAR(1),
   IN p_alamat VARCHAR(100),
   IN p_hp VARCHAR(15),
   OUT p_kode_dihasilkan CHAR(5)
)
BEGIN
   DECLARE last_num INT DEFAULT 0;

   -- Hitung urutan nomor ID customer terakhir (Format M0001, M0002, dst)
   SELECT IFNULL(MAX(CAST(RIGHT(kodecus, 4) AS UNSIGNED)), 0) INTO last_num FROM customer;
   SET p_kode_dihasilkan = CONCAT('M', LPAD(last_num + 1, 4, '0'));

   -- Simpan data fisik ke tabel customer
   INSERT INTO customer (kodecus, namacus, tgllahir, jenkelcus, almtcus, hpcus, pointterkumpul)
   VALUES (p_kode_dihasilkan, p_nama, p_tgllahir, p_jenkel, p_alamat, p_hp, 0);
END$$

-- Procedure B: Parameter INOUT (Menghitung Tambahan Poin Loyalitas)
CREATE PROCEDURE spHitungBonusLoyalitas(
   IN p_status_member VARCHAR(20),
   INOUT p_poin INT
)
BEGIN
   IF p_status_member = 'VIP' THEN
      SET p_poin = p_poin + 20; -- Bonus tambahan 20 poin
```

```
  ELSEIF p_status_member = 'REGULER' THEN
    SET p_poin = p_poin + 5;  -- Bonus tambahan 5 poin
  END IF;
END$$

DELIMITER ;

-- Pengujian Eksekusi Parameter OUT:
CALL spRegistrasiMember('Rina Melati', '1998-07-21', 'P', 'Yogyakarta', '081299887766', @kode_baru);
SELECT @kode_baru AS 'ID Member Terdaftar';

-- Pengujian Eksekusi Parameter INOUT:
SET @poin_awal = 10;
CALL spHitungBonusLoyalitas('VIP', @poin_awal);
SELECT @poin_awal AS 'Total Poin Setelah Bonus VIP'; -- Output: 30 (10 masuk, bertambah 20, kembali 30)
```

6. KONTROL ALUR LOGIKA (CONTROL FLOW STATEMENTS)

Salah satu ciri pembeda mendasar antara Stored Routine dan View adalah keberadaan struktur kendali alur (Control Flow). Bagi mahasiswa yang sudah memahami dasar pemrograman (seperti Python, Java, atau C++), struktur ini memiliki logika yang identik, hanya saja tempat eksekusinya berada di dalam mesin database:

- **1. Pernyataan IF-THEN-ELSEIF-ELSE:** Percabangan bersyarat bertingkat: IF kondisi THEN ... ELSEIF kondisi THEN ... ELSE ... END IF;
- **2. Pernyataan CASE:** Percabangan pembanding banyak nilai: CASE ekspresi WHEN nilai1 THEN ... WHEN nilai2 THEN ... ELSE ... END CASE;
- **3. Perulangan WHILE-DO:** Perulangan dengan evaluasi kondisi di awal: WHILE kondisi DO ... instruksi ... END WHILE; (perulangan berhenti jika kondisi bernilai FALSE).
- **4. Perulangan REPEAT-UNTIL:** Perulangan dengan evaluasi kondisi di akhir: REPEAT ... instruksi ... UNTIL kondisi END REPEAT; (instruksi dijamin dieksekusi minimal satu kali).
- **5. Perulangan LOOP-LEAVE:** Perulangan tanpa henti yang dihentikan secara eksplisit menggunakan perintah LEAVE: [label:] LOOP ... IF kondisi THEN LEAVE [label]; END IF; END LOOP [label];

7. CURSOR & ITERASI DATA BARIS DEMI BARIS

Secara default, seluruh operasi kueri SQL bekerja berbasis himpunan (set-based). Contohnya, perintah 'UPDATE customer SET pointterkumpul = pointterkumpul + 10' akan memproses seluruh baris customer secara serentak dalam satu hentakan. Namun, bagaimana jika manajemen menginginkan aturan perhitungan yang berbeda-beda untuk tiap customer berdasarkan riwayat transaksi individual mereka? Di sinilah sistem basis data membutuhkan CURSOR.

> 💡 **Analogi Membaca Daftar Absensi:** *Cursor ibarat dosen yang membaca lembar daftar absensi mahasiswa satu per satu:*
> *1. Dosen memegang lembaran daftar nama mahasiswa (DECLARE CURSOR FOR SELECT...).*
> *2. Menyiapkan penanda jika daftar sudah habis (DECLARE CONTINUE HANDLER FOR NOT FOUND...).*
> *3. Membuka lembaran absensi (OPEN curCustomer).*
> *4. Menunjuk nama pertama, membaca datanya (FETCH INTO), memutuskan aksi nilai, lalu berpindah ke nama berikutnya (LOOP).*
> *5. Begitu mencapai baris paling bawah, dosen selesai dan menutup map absensi (CLOSE curCustomer).*

7.1 Lima Siklus Wajib Implementasi Cursor

- **1. DECLARE CURSOR:** Mendefinisikan kueri SQL pembentuk baris yang akan dibaca: DECLARE cur CURSOR FOR SELECT kolom1, kolom2 FROM tabel;
- **2. DECLARE HANDLER NOT FOUND:** Mendaftarkan penangan kondisi ketika baris kursor telah habis: DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_selesai = 1;
- **3. OPEN CURSOR:** Mengalokasikan memori internal server dan mengeksekusi kueri kursor: OPEN cur;
- **4. FETCH & LOOP:** Mengambil nilai kolom pada baris aktif ke dalam variabel lokal: FETCH cur INTO var1, var2; lalu keluar loop jika v_selesai = 1.
- **5. CLOSE CURSOR:** Membebaskan memori kerja kursor dari RAM server: CLOSE cur;

> ⚠️ **Catatan Kritis Kinerja (Performance Trade-off):** *Mengapa Cursor terasa lambat untuk dataset besar? Cursor memproses data secara sekuensial (row-by-row), sehingga memerlukan alokasi memori dan context-switching CPU per baris. Jika suatu transformasi data dapat diselesaikan menggunakan kueri set-based murni (seperti UPDATE dengan CASE-WHEN atau INSERT-SELECT), maka pendekatan set-based jauh lebih disarankan. Cursor digunakan hanya bila tiap baris memerlukan pemanggilan subrutin terpisah atau evaluasi prosedural bertingkat.*

7.2 Contoh Nyata: Pembagian Bonus Poin Massal Customer

```
DELIMITER $$

CREATE PROCEDURE spProsesBonusPoinMassal()
BEGIN
  DECLARE v_kodecus CHAR(5);
  DECLARE v_poin_saat_ini INT;
  DECLARE v_selesai INT DEFAULT 0;

  -- 1. Deklarasi Cursor untuk memilih customer (Kecuali customer umum M0000)
  DECLARE curCustomer CURSOR FOR
    SELECT kodecus, pointterkumpul
    FROM customer
    WHERE kodecus <> 'M0000';

  -- 2. Deklarasi Handler NOT FOUND (Dipicu saat FETCH sudah tidak menemukan baris lagi)
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_selesai = 1;

  -- 3. Membuka Cursor
  OPEN curCustomer;

  loopCustomer: LOOP
    -- 4. Mengambil satu baris data ke dalam variabel
    FETCH curCustomer INTO v_kodecus, v_poin_saat_ini;

    -- Cek apakah data sudah habis
    IF v_selesai = 1 THEN
      LEAVE loopCustomer;
    END IF;

    -- Evaluasi aturan bisnis berjenjang per baris
    IF v_poin_saat_ini >= 100 THEN
      UPDATE customer SET pointterkumpul = pointterkumpul + 20 WHERE kodecus = v_kodecus;
```

```
    ELSEIF v_poin_saat_ini > 50 THEN
      UPDATE customer SET pointterkumpul = pointterkumpul + 15 WHERE kodecus = v_kodecus;
    ELSEIF v_poin_saat_ini > 20 THEN
      UPDATE customer SET pointterkumpul = pointterkumpul + 10 WHERE kodecus = v_kodecus;
    ELSE
      UPDATE customer SET pointterkumpul = pointterkumpul + 7 WHERE kodecus = v_kodecus;
    END IF;
  END LOOP loopCustomer;

  -- 5. Menutup Cursor
  CLOSE curCustomer;

  SELECT 'Pemrosesan bonus poin seluruh member selesai dieksekusi.' AS status_eksekusi;
END$$

DELIMITER ;

-- Eksekusi Prosedur:
CALL spProsesBonusPoinMassal();
```

8. PENANGANAN GALAT (ERROR HANDLING) & INTEGRITAS TRANSAKSI ACID

Bayangkan skenario kasir toko ritel: Kasir sedang memproses belanja pelanggan. Stok produk di tabel PRODUK telah dipotong, namun tiba-tiba koneksi jaringan putus atau listrik mati sebelum nomor nota berhasil dicatat ke tabel NOTA. Tanpa penanganan galat dan kontrol transaksi yang tepat, basis data akan mengalami inkonsistensi parah: barang fisik toko berkurang tanpa adanya bukti riwayat transaksi penjualan.
Untuk mencegah anomali tersebut, MySQL menyediakan mekanisme penanganan eksepsi terstruktur:

> 💡 **Analogi Satpam Penjaga Toko & Mesin ATM:** • *SIGNAL SQLSTATE '45000' ibarat SATPAM PENJAGA TOKO: Saat ada pembeli ingin membeli 5 barang padahal stok tersisa 2, satpam langsung menghentikan transaksi dan memberikan penjelasan yang jelas kepada pembeli: 'Transaksi ditolak: Stok tidak cukup!'. Kode '45000' adalah kode resmi internasional untuk kesalahan yang didefinisikan sendiri oleh pengguna (User-Defined Exception).*
> • *TRANSAKSI (START TRANSACTION, COMMIT, ROLLBACK) ibarat TRANSFER UANG DI ATM: Uang ditarik dari rekening pengirim DAN dimasukkan ke rekening penerima. Keduanya WAJIB berhasil bersama-sama (COMMIT). Jika mesin ATM macet di tengah jalan, seluruh proses dibatalkan dan saldo pengirim dikembalikan utuh (ROLLBACK). Tidak boleh ada kondisi setengah jalan.*

8.1 Contoh Implementasi: Validasi Stok dengan SIGNAL SQLSTATE

```
DELIMITER $$

CREATE PROCEDURE spKurangiStokBarang(
  IN p_kodeproduk CHAR(4),
  IN p_qty INT
)
BEGIN
  DECLARE v_stok_tersedia INT;

  -- Ambil persediaan stok barang saat ini
  SELECT stok INTO v_stok_tersedia FROM produk WHERE kodeproduk = p_kodeproduk;
```

```
    -- Validasi 1: Periksa apakah kode produk terdaftar
    IF v_stok_tersedia IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Kesalahan: Kode produk barang tidak ditemukan dalam database.';
    END IF;

    -- Validasi 2: Periksa kecukupan kuantitas stok
    IF v_stok_tersedia < p_qty THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Transaksi Dibatalkan: Stok produk di gudang tidak mencukupi.';
    ELSE
        -- Update pemotongan stok fisik
        UPDATE produk SET stok = stok - p_qty WHERE kodeproduk = p_kodeproduk;
    END IF;
END$$

DELIMITER ;

-- Pengujian 1: Stok Cukup (Berhasil)
CALL spKurangiStokBarang('P001', 2);

-- Pengujian 2: Stok Melebihi Batas (Sistem melempar Error 45000)
CALL spKurangiStokBarang('P001', 9999);
-- Respon Konsol: ERROR 1644 (45000): Transaksi Dibatalkan: Stok produk di gudang tidak mencukupi.
```

9. STUDI KASUS PENUTUP: PROSEDUR TRANSAKSI CHECKOUT KASIR TERPADU

Prosedur spCheckoutTransaksi di bawah ini berfungsi sebagai studi kasus terpadu transaksi minimarket. Prosedur ini menggabungkan seluruh konsep yang telah dipelajari sebelumnya ke dalam satu alur transaksi kasir nyata:

- Memanggil UDF (fcGenerateNoNota dan fcHitungPoinBelanja)
- Menggunakan parameter input (IN) dan parameter hasil keluaran (OUT)
- Melakukan validasi integritas data dengan SIGNAL SQLSTATE '45000'
- Membungkus mutasi multi-tabel dalam transaksi atomik (START TRANSACTION - COMMIT)
- Menerapkan proteksi pembatalan otomatis dengan DECLARE EXIT HANDLER FOR SQLEXCEPTION (ROLLBACK).

```
DELIMITER $$

CREATE PROCEDURE spCheckoutTransaksi(
    IN p_kodekasir CHAR(3),
    IN p_kodecus CHAR(5),
    IN p_kodeproduk CHAR(4),
    IN p_banyak INT,
    OUT p_nonota_dihasilkan CHAR(10),
    OUT p_total_bayar INT,
    OUT p_poin_baru INT
)
BEGIN
    DECLARE v_harga_satuan INT;
    DECLARE v_stok_tersedia INT;
    DECLARE v_tgl_hari_ini DATE;
```

```
    -- Penanganan galat otomatis: Rollback semua mutasi tabel jika terjadi galat sistem
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    SET v_tgl_hari_ini = CURRENT_DATE();

    -- 1. Validasi Keberadaan Data Produk & Persediaan Stok
    SELECT harga, stok INTO v_harga_satuan, v_stok_tersedia
    FROM produk WHERE kodeproduk = p_kodeproduk;

    IF v_harga_satuan IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Produk tidak terdaftar.';
    END IF;

    IF v_stok_tersedia < p_banyak THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Kuantitas beli melebihi persediaan stok toko.';
    END IF;

    -- 2. Kalkulasi Nilai & Pemanggilan UDF
    SET p_nonota_dihasilkan = fcGenerateNoNota(v_tgl_hari_ini);
    SET p_total_bayar = v_harga_satuan * p_banyak;
    SET p_poin_baru = fcHitungPoinBelanja(p_total_bayar);

    -- 3. Memulai Blok Transaksi Atomik (ACID)
    START TRANSACTION;
        -- A. Simpan Header Nota Penjualan
        INSERT INTO nota (nonota, kodekasir, tgltran, kodecus)
        VALUES (p_nonota_dihasilkan, p_kodekasir, v_tgl_hari_ini, p_kodecus);

        -- B. Simpan Rincian Item Transaksi
        INSERT INTO itemnota (nonota, kodeproduk, banyak, hargajual)
        VALUES (p_nonota_dihasilkan, p_kodeproduk, p_banyak, v_harga_satuan);

        -- C. Kurangi Stok Fisik Produk
        UPDATE produk SET stok = stok - p_banyak WHERE kodeproduk = p_kodeproduk;

        -- D. Tambahkan Poin Belanja ke Akun Member (Kecuali customer umum M0000)
        IF p_kodecus <> 'M0000' AND p_poin_baru > 0 THEN
            UPDATE customer SET pointterkumpul = pointterkumpul + p_poin_baru WHERE kodecus = p_kodecus;
        END IF;
    COMMIT;
END$$

DELIMITER ;

-- Pengujian Skenario Transaksi Kasir:
CALL spCheckoutTransaksi('001', 'M0001', 'P001', 2, @nota, @total, @poin);
SELECT @nota AS 'Nomor Nota', @total AS 'Total Bayar (Rp)', @poin AS 'Poin Diperoleh';
```

10. PANDUAN PEDAGOGIS & METODOLOGI MENGAJAR (KHUSUS DOSEN & ASPRAK)

Berdasarkan evaluasi pengajaran praktikum SMBD Lanjut di laboratorium, materi Stored Routine sering kali menjadi titik peralihan yang menantang bagi mahasiswa karena menggabungkan pemikiran aljabar relasional dengan pemikiran algoritma algoritmik. Berikut adalah 7 langkah urutan mengajar terstruktur yang disarankan:

- **Langkah 1: Jembatan Konseptual (View -> Routine):** Mulai dengan menunjukkan perbedaan antara View (hanya satu SELECT yang diberi nama) dengan Stored Routine (program utuh yang memiliki logika di dalam basis data).
- **Langkah 2: Mulai dari Function (UDF):** Ajarkan Function terlebih dahulu karena konsepnya paling mirip dengan query SQL yang sudah dikenal mahasiswa (hasilnya langsung dapat dilihat di dalam SELECT).
- **Langkah 3: Alasan Penggunaan DELIMITER:** Jelaskan alasan penggantian tanda akhir perintah menjadi '$$' saat mendemonstrasikan pembuatan Function pertama, agar mahasiswa tidak bingung dengan Syntax Error.
- **Langkah 4: Stored Procedure & Parameter (IN, OUT, INOUT):** Gunakan analogi mesin fotokopi untuk membedakan aliran data parameter IN (dokumen asli), OUT (hasil fotokopi), dan INOUT (formulir yang dicoret-coret). Tekankan pentingnya tanda '@' untuk variabel sesi penampung.
- **Langkah 5: Struktur Kendali Logika (Control Flow):** Tunjukkan bahwa sintaks IF dan WHILE pada dasarnya sama seperti di bahasa pemrograman yang telah dipelajari mahasiswa (Python/Java), hanya berbeda tempat eksekusinya.
- **Langkah 6: Cursor & Iterasi Baris Demi Baris:** Gunakan analogi membaca daftar absensi mahasiswa. Arahkan mahasiswa memahami 5 tahapan berurutan: DECLARE CURSOR -> DECLARE HANDLER NOT FOUND -> OPEN -> FETCH di LOOP -> CLOSE. Jelaskan fungsi variabel bendera v_selesai.
- **Langkah 7: Error Handling & Transaksi ACID:** Gunakan skenario listrik padam di kasir untuk menjelaskan mengapa transaksi atomik (START TRANSACTION - COMMIT/ROLLBACK) dan SIGNAL 45000 sangat penting untuk menjaga integritas data.

11. GLOSARIUM & RINGKASAN ISTILAH KUNCI

Tabel referensi cepat istilah-istilah esensial untuk pegangan belajar praktikan dan evaluasi laboratorium:

Istilah / Perintah	Definisi Singkat & Peran Teknis
Stored Routine	Program prosedural berlogika yang disimpan permanen di dalam katalog basis data.
User Defined Function (UDF)	Subrutin yang wajib mengembalikan satu nilai skalar via RETURN, dipanggil di dalam SELECT.
Stored Procedure	Subrutin untuk mengerjakan alur tugas bisnis mandiri, dipanggil dengan perintah CALL.
DELIMITER $$	Perintah untuk mengubah karakter penutup instruksi sementara agar titik-koma di blok BEGIN-END tidak memutus kueri.
Parameter IN	Mode masukan nilai ke dalam procedure (default); procedure membaca nilai tanpa mengubah aslinya.
Parameter OUT	Mode keluaran; procedure mengisi nilai hasil komputasi untuk diambil kembali oleh pemanggil.
Parameter INOUT	Mode dua arah; membawa nilai awal masuk, diperbarui di dalam procedure, lalu dikembalikan.
Variabel Sesi (@variabel)	Variabel tingkat koneksi pengguna di MySQL yang digunakan untuk menampung nilai parameter OUT/INOUT.
CURSOR	Penunjuk memori internal untuk membaca dan memproses baris

	data hasil kueri satu per satu.
NOT FOUND Handler	Penangan otomatis saat kursor mencapai baris terakhir (digunakan untuk memicu keluar loop).
SIGNAL SQLSTATE '45000'	Perintah untuk melempar pesan kesalahan kustom buatan pengguna ke aplikasi klien.
Transaksi ACID	Jaminan atomisitas bahwa seluruh perubahan multi-tabel berhasil bersama (COMMIT) atau batal bersama (ROLLBACK).

12. LEMBAR TUGAS & LATIHAN SOAL PRAKTIKUM (UNTUK MAHASISWA)

Petunjuk Pengerjaan Mahasiswa / Praktikan:
1. Pastikan skema basis data minimarket praktikum (PRODUK, KASIR, CUSTOMER, NOTA, ITEMNOTA) telah diimpor dan aktif.
2. Tuliskan kueri pembuatan Function atau Procedure secara lengkap dengan penyesuaian DELIMITER.
3. Lakukan pengujian eksekusi (CALL atau SELECT) dan tangkap layar (screenshot) kode kueri serta respon output terminal.
4. Perhatikan mode parameter (IN, OUT, INOUT) dan penanganan kondisi error batas (seperti nilai NULL atau stok 0).

Tingkat 1: User Defined Function (UDF) & Logika Kalkulasi

Soal 1 (UDF Evaluasi Persediaan Stok): Buatlah UDF bernama 'fcCekStatusStok' yang menerima parameter 'p_kodeproduk' (CHAR(4)) dan mengembalikan status stok (VARCHAR(30)). Ketentuan: jika stok = 0 kembalikan 'KOSONG', jika stok 1 s.d. 5 kembalikan 'KRITIS', jika lebih dari 5 kembalikan 'TERSEDIA (X Unit)', dan jika kode produk tidak ada kembalikan 'PRODUK TIDAK ADA'. Uji dengan menampilkan daftar produk dan status stoknya.

Soal 2 (UDF Klasifikasi Member Loyal): Toko ingin mengklasifikasikan jenjang keanggotaan pelanggan berdasarkan perolehan poin. Buatlah UDF bernama 'fcLevelLoyalitas' yang menerima input 'p_point' (INT) dan mengembalikan string status loyalitas: Point >= 100 'PLATINUM', Point 50 s.d. 99 'GOLD', Point 20 s.d. 49 'SILVER', dan Point < 20 'BRONZE'. Tampilkan seluruh data pelanggan beserta tier loyalitasnya menggunakan fungsi ini.

Soal 3 (UDF Pemformatan Teks Rupiah): Buatlah UDF bernama 'fcFormatRupiah' yang menerima input nominal angka (INT) dan mengembalikan string berformat mata uang Indonesia dengan awalan 'Rp. ' dan pemisah ribuan titik (contoh: angka 15000 menjadi string 'Rp. 15.000'). Uji dengan menampilkan harga produk.

Tingkat 2: Stored Procedure dengan Parameter IN, OUT, dan INOUT

Soal 4 (Procedure Parameter OUT Informasi Customer): Buatlah Stored Procedure bernama 'spCariDataCustomer' dengan satu parameter IN (p_kodecus CHAR(5)) dan tiga parameter OUT (p_nama VARCHAR(50), p_alamat VARCHAR(100), p_poin INT). Procedure mencari customer bersangkutan dan menyimpan nilainya ke ketiga parameter OUT. Tuliskan kueri pemanggilan dengan CALL dan tampilkan isi variabel penampungnya.

Soal 5 (Procedure Parameter INOUT Diskon Harga): Manajemen minimarket ingin melakukan diskon persentase harga barang. Buatlah Stored Procedure bernama 'spTerapkanDiskonProduk' yang memiliki parameter IN 'p_kodeproduk' (CHAR(4)) dan parameter INOUT 'p_harga' (INT). Nilai harga lama yang dilewatkan ke parameter akan dipotong sebesar 10%, kemudian nilai harga baru tersebut diperbarui ke tabel PRODUK dan nilainya dikembalikan ke pemanggil.

Soal 6 (Procedure Registrasi Staf Kasir): Buatlah Stored Procedure bernama 'spTambahKasirBaru' yang menerima parameter IN nama kasir dan password kasir. Di dalam procedure, generate kode kasir 3 digit otomatis (contoh: '003', '004') dan simpan ke tabel KASIR. Tampilkan data kasir baru yang berhasil ditambahkan.

Tingkat 3: Cursor & Batch Processing Transaksi

Soal 7 (Cursor Kompilasi Riwayat Nota Belanja): Buatlah Stored Procedure bernama 'spKompilasiBelanjaMember' yang memanfaatkan CURSOR untuk mengiterasi seluruh riwayat transaksi belanja seorang customer tertentu (input parameter kodecus). Gabungkan nomor-nomor nota belanja yang pernah dilakukan customer tersebut ke dalam satu variabel string panjang yang dipisahkan oleh tanda koma, lalu tampilkan hasilnya.

Soal 8 (Cursor Pembaruan Stok Masal): Toko ingin merestok seluruh barang yang memiliki stok di bawah 10 unit. Buatlah Stored Procedure 'spRestokOtomatis' yang menggunakan CURSOR untuk mencari semua produk berkategori 'stok kritis' (< 10) dan secara otomatis menambahkan 50 unit ke masing-masing produk tersebut. Catat dan tampilkan berapa banyak produk yang berhasil direstok.

Tingkat 4: Error Handling & Complex Transaction Control

Soal 9 (Validasi Integritas Hapus Data dengan SIGNAL): Buatlah Stored Procedure 'spHapusCustomerAman' yang menerima kode customer. Procedure harus memvalidasi: jika customer tersebut pernah memiliki riwayat transaksi pada tabel NOTA,

batalkan operasi dan lemparkan error kustom via SIGNAL SQLSTATE '45000' dengan pesan: 'Tidak dapat menghapus customer: Data customer terikat dengan riwayat nota transaksi'. Jika tidak memiliki nota, lakukan penghapusan.

Soal 10 (Prosedur Transaksi Atomik Penjualan Barang): Rancanglah Stored Procedure 'spPencatatanItemTransaksi' yang menerima parameter: nomor nota, kode produk, dan jumlah beli. Gunakan HANDLER FOR SQLEXCEPTION untuk menjamin atomisitas data. Di dalam procedure: periksa kecukupan stok di tabel PRODUK, kurangi stok barang, dan simpan baris transaksi ke tabel ITEMNOTA. Jika stok tidak cukup, batalkan transaksi dan lempar error kustom.

13. LAMPIRAN: KUNCI JAWABAN & PEMBAHASAN LENGKAP (KHUSUS DOSEN & ASISTEN PRAKTIKUM)

Bagian ini memuat seluruh solusi kueri SQL baku, sintaks pengujian eksekusi, serta analisis logika teknis sebagai pedoman evaluasi dan rubrik penilaian bagi dosen dan asisten praktikum:

Solusi & Pembahasan Soal 1 (fcCekStatusStok)

```
DELIMITER $$
CREATE FUNCTION fcCekStatusStok(p_kodeproduk CHAR(4))
RETURNS VARCHAR(30)
DETERMINISTIC
READS SQL DATA
BEGIN
   DECLARE v_stok INT;
   DECLARE v_status VARCHAR(30);

   SELECT stok INTO v_stok FROM produk WHERE kodeproduk = p_kodeproduk;

   IF v_stok IS NULL THEN
      SET v_status = 'PRODUK TIDAK ADA';
   ELSEIF v_stok = 0 THEN
      SET v_status = 'KOSONG';
   ELSEIF v_stok <= 5 THEN
      SET v_status = 'KRITIS';
   ELSE
      SET v_status = CONCAT('TERSEDIA (', v_stok, ' Unit)');
   END IF;

   RETURN v_status;
END$$
DELIMITER ;

-- Kueri Pengujian:
SELECT kodeproduk, namaproduk, stok, fcCekStatusStok(kodeproduk) AS status_stok FROM produk;
```

Analisis Logika: Analisis: Fungsi membaca data stok dari tabel produk dan mengevaluasi ambang batas menggunakan struktur IF-ELSEIF secara deterministik.

Solusi & Pembahasan Soal 2 (fcLevelLoyalitas)

```
DELIMITER $$
CREATE FUNCTION fcLevelLoyalitas(p_point INT)
RETURNS VARCHAR(15)
DETERMINISTIC
NO SQL
BEGIN
   DECLARE v_level VARCHAR(15);
   IF p_point >= 100 THEN
      SET v_level = 'PLATINUM';
   ELSEIF p_point >= 50 THEN
      SET v_level = 'GOLD';
   ELSEIF p_point >= 20 THEN
      SET v_level = 'SILVER';
```

```
    ELSE
        SET v_level = 'BRONZE';
    END IF;
    RETURN v_level;
END$$
DELIMITER ;

-- Kueri Pengujian:
SELECT kodecus, namacus, pointterkumpul, fcLevelLoyalitas(pointterkumpul) AS tier_loyalitas FROM customer;
```

Analisis Logika: Analisis: Menggunakan karakteristik NO SQL karena murni komputasi skalar tanpa mengakses data tabel fisik.

Solusi & Pembahasan Soal 3 (fcFormatRupiah)

```
DELIMITER $$
CREATE FUNCTION fcFormatRupiah(p_nominal INT)
RETURNS VARCHAR(30)
DETERMINISTIC
NO SQL
BEGIN
    RETURN CONCAT('Rp. ', REPLACE(FORMAT(p_nominal, 0), ',', '.'));
END$$
DELIMITER ;

-- Kueri Pengujian:
SELECT kodeproduk, namaproduk, fcFormatRupiah(harga) AS harga_rupiah FROM produk;
```

Analisis Logika: Analisis: Memanfaatkan fungsi bawaan FORMAT() dan REPLACE() untuk mengubah koma standar internasional menjadi titik ribuan khas Indonesia.

Solusi & Pembahasan Soal 4 (spCariDataCustomer)

```
DELIMITER $$
CREATE PROCEDURE spCariDataCustomer(
    IN p_kodecus CHAR(5),
    OUT p_nama VARCHAR(50),
    OUT p_alamat VARCHAR(100),
    OUT p_poin INT
)
BEGIN
    SELECT namacus, almtcus, pointterkumpul
    INTO p_nama, p_alamat, p_poin
    FROM customer
    WHERE kodecus = p_kodecus;
END$$
DELIMITER ;

-- Kueri Pengujian:
CALL spCariDataCustomer('M0001', @nama, @alamat, @poin);
SELECT @nama AS 'Nama', @alamat AS 'Alamat', @poin AS 'Poin Terkumpul';
```

Analisis Logika: Analisis: Menggunakan SELECT INTO untuk menampung data baris tunggal ke dalam variabel parameter bertipe OUT.

Solusi & Pembahasan Soal 5 (spTerapkanDiskonProduk)

```
DELIMITER $$
CREATE PROCEDURE spTerapkanDiskonProduk(
    IN p_kodeproduk CHAR(4),
    INOUT p_harga INT
)
BEGIN
    -- Potong harga sebesar 10%
    SET p_harga = p_harga * 0.90;

    -- Perbarui harga di tabel master produk
    UPDATE produk SET harga = p_harga WHERE kodeproduk = p_kodeproduk;
END$$
DELIMITER ;

-- Kueri Pengujian:
SET @harga_produk = 10000;
CALL spTerapkanDiskonProduk('P001', @harga_produk);
SELECT @harga_produk AS 'Harga Baru Setelah Diskon 10%'; -- Output: 9000
```

Analisis Logika: Analisis: Mendemonstrasikan parameter INOUT yang membawa harga awal masuk, dipotong oleh procedure, diperbarui ke tabel, dan nilainya dikembalikan ke pemanggil.

Solusi & Pembahasan Soal 6 (spTambahKasirBaru)

```
DELIMITER $$
CREATE PROCEDURE spTambahKasirBaru(
    IN p_namakasir VARCHAR(50),
    IN p_passwordkasir VARCHAR(50)
)
BEGIN
    DECLARE v_max_id INT DEFAULT 0;
    DECLARE v_new_kode CHAR(3);

    -- Cari nomor urut kasir terbesar (Format '001', '002', dst)
    SELECT IFNULL(MAX(CAST(kodekasir AS UNSIGNED)), 0) INTO v_max_id FROM kasir;
    SET v_new_kode = LPAD(v_max_id + 1, 3, '0');

    -- Insert staf kasir baru
    INSERT INTO kasir (kodekasir, namakasir, passwordkasir)
    VALUES (v_new_kode, p_namakasir, p_passwordkasir);

    -- Tampilkan kasir yang baru dibuat
    SELECT * FROM kasir WHERE kodekasir = v_new_kode;
END$$
DELIMITER ;
```

```
-- Kueri Pengujian:
CALL spTambahKasirBaru('Budi Santoso', 'kasir123');
```

Analisis Logika: Analisis: Menggunakan fungsi CAST dan LPAD untuk membentuk format string kode numerik berpadding tiga digit secara dinamis.

Solusi & Pembahasan Soal 7 (spKompilasiBelanjaMember)

```
DELIMITER $$
CREATE PROCEDURE spKompilasiBelanjaMember(IN p_kodecus CHAR(5))
BEGIN
   DECLARE v_nonota CHAR(10);
   DECLARE v_selesai INT DEFAULT 0;
   DECLARE v_daftar_nota TEXT DEFAULT '';

   -- 1. Deklarasi Cursor untuk mengiterasi nota milik customer bersangkutan
   DECLARE curNota CURSOR FOR
      SELECT nonota FROM nota WHERE kodecus = p_kodecus ORDER BY tgltran ASC;

   -- 2. Deklarasi Handler NOT FOUND
   DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_selesai = 1;

   OPEN curNota;

   loopNota: LOOP
      FETCH curNota INTO v_nonota;
      IF v_selesai = 1 THEN
         LEAVE loopNota;
      END IF;

      IF v_daftar_nota = '' THEN
         SET v_daftar_nota = v_nonota;
      ELSE
         SET v_daftar_nota = CONCAT(v_daftar_nota, ', ', v_nonota);
      END IF;
   END LOOP loopNota;

   CLOSE curNota;

   SELECT p_kodecus AS customer, v_daftar_nota AS riwayat_nota;
END$$
DELIMITER ;

-- Kueri Pengujian:
CALL spKompilasiBelanjaMember('M0001');
```

Analisis Logika: Analisis: Menggunakan CURSOR untuk mengagregasi baris data relasional ke dalam satu representasi teks berpemisah koma.

Solusi & Pembahasan Soal 8 (spRestokOtomatis)

```
DELIMITER $$
CREATE PROCEDURE spRestokOtomatis()
BEGIN
   DECLARE v_kodeproduk CHAR(4);
   DECLARE v_selesai INT DEFAULT 0;
   DECLARE v_count INT DEFAULT 0;

   -- 1. Deklarasi Cursor untuk produk berstok kritis (< 10 unit)
   DECLARE curKritis CURSOR FOR
      SELECT kodeproduk FROM produk WHERE stok < 10;

   -- 2. Deklarasi Handler NOT FOUND
   DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_selesai = 1;

   OPEN curKritis;

   loopRestok: LOOP
      FETCH curKritis INTO v_kodeproduk;
      IF v_selesai = 1 THEN
         LEAVE loopRestok;
      END IF;

      -- Tambahkan 50 unit ke stok fisik
      UPDATE produk SET stok = stok + 50 WHERE kodeproduk = v_kodeproduk;
      SET v_count = v_count + 1;
   END LOOP loopRestok;

   CLOSE curKritis;

   SELECT CONCAT('Restok otomatis selesai. Jumlah produk yang ditambah stok: ', v_count) AS laporan_restok;
END$$
DELIMITER ;

-- Kueri Pengujian:
CALL spRestokOtomatis();
```

Analisis Logika: Analisis: Mengotomatisasi penambahan inventori produk kritis dan menghitung akumulasi total baris yang berhasil termutasi.

Solusi & Pembahasan Soal 9 (spHapusCustomerAman)

```
DELIMITER $$
CREATE PROCEDURE spHapusCustomerAman(IN p_kodecus CHAR(5))
BEGIN
   DECLARE v_jml_nota INT DEFAULT 0;

   -- Periksa apakah customer terikat dengan riwayat nota
   SELECT COUNT(*) INTO v_jml_nota FROM nota WHERE kodecus = p_kodecus;

   IF v_jml_nota > 0 THEN
      SIGNAL SQLSTATE '45000'
```

```
    SET MESSAGE_TEXT = 'Tidak dapat menghapus customer: Data customer terikat dengan riwayat nota transaksi.';
  ELSE
    DELETE FROM customer WHERE kodecus = p_kodecus;
    SELECT CONCAT('Customer ', p_kodecus, ' berhasil dihapus.') AS status;
  END IF;
END$$
DELIMITER ;

-- Kueri Pengujian (Customer yang memiliki nota M0001 -> Ditolak via SIGNAL):
CALL spHapusCustomerAman('M0001');
-- Respon: ERROR 1644 (45000): Tidak dapat menghapus customer: Data customer terikat dengan riwayat nota transaksi.
```

Analisis Logika: Analisis: Mencegah galat foreign key cascaded yang tidak terkontrol dengan menerapkan validasi logika ramah pengguna via SIGNAL SQLSTATE '45000'.

Solusi & Pembahasan Soal 10 (spPencatatanItemTransaksi)

```
DELIMITER $$
CREATE PROCEDURE spPencatatanItemTransaksi(
  IN p_nonota CHAR(10),
  IN p_kodeproduk CHAR(4),
  IN p_qty INT
)
BEGIN
  DECLARE v_stok INT DEFAULT 0;
  DECLARE v_harga INT DEFAULT 0;

  -- Rollback otomatis jika terjadi eksepsi database
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    RESIGNAL;
  END;

  SELECT stok, harga INTO v_stok, v_harga FROM produk WHERE kodeproduk = p_kodeproduk;

  IF v_stok IS NULL THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Produk tidak ditemukan.';
  END IF;

  IF v_stok < p_qty THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: Kuantitas melebihi ketersediaan stok fisik toko.';
  END IF;

  START TRANSACTION;
    INSERT INTO itemnota (nonota, kodeproduk, banyak, hargajual)
    VALUES (p_nonota, p_kodeproduk, p_qty, v_harga);

    UPDATE produk SET stok = stok - p_qty WHERE kodeproduk = p_kodeproduk;
  COMMIT;

  SELECT 'Item nota berhasil dicatat dan stok fisik terpotong.' AS status;
```

```
END$$
DELIMITER ;
```

Analisis Logika: Analisis: Menggunakan blok transaksi START TRANSACTION - COMMIT dengan jaminan atomisitas via DECLARE EXIT HANDLER FOR SQLEXCEPTION.

14. DAFTAR PUSTAKA & REFERENSI RESMI

- **1.** Oracle Corporation. (2024). MySQL 8.0 Reference Manual: Stored Routines & Cursor Management. https://dev.mysql.com/doc/refman/8.0/en/stored-routines.html
- **2.** Carlos Coronel & Steven Morris. (2019). Database Systems: Design, Implementation, & Management (13th Edition). Cengage Learning, Boston, USA.
- **3.** Paul DuBois. (2014). MySQL Developer's Library (5th Edition). Addison-Wesley Professional.
- **4.** Laboratorium Sistem Informasi. (2024). Modul Praktikum Sistem Manajemen Basis Data Lanjut: Materi Praktikum 4. Universitas Amikom Yogyakarta.